

Midterm Project Report

Sliding-window CNN Accelerator

Willie Liao – GEL8580
 Peixin Zhang – JZH2008
 May 18, 2026

1 Project Overview

This report the midterm progress of A fixed-point, MobileFaceNet-specific convolution frontend accelerator. Rather than implementing a fully configurable NPU, the design focuses on executing the convolutional frontend of MobileFaceNet with a fixed-point datapath and layer-by-layer control flow.

2 Model Design & Precision Selection

2.1 MobileFaceNet Architecture

We target the MobileFaceNet variant with input size $3 \times 112 \times 96$. The original MobileFaceNet paper [1] and the hardware implementation in [2] use a square input of $3 \times 112 \times 112$ with a 7×7 global depthwise convolution. In this work, we adopt the non-square variant from [3], which replaces the GDConv kernel with 7×6 to match the reduced horizontal resolution, as summarized in Table 1.

Table 1: MobileFaceNet Layer Configuration ($3 \times 112 \times 96$ input)

Input Shape	Operator	t	c	n	s
$3 \times 112 \times 96$	Conv 3×3	–	64	1	2
$64 \times 56 \times 48$	DW Conv 3×3	–	64	1	1
$64 \times 56 \times 48$	Bottleneck	2	64	5	2
$64 \times 28 \times 24$	Bottleneck	4	128	1	2
$128 \times 14 \times 12$	Bottleneck	2	128	6	1
$128 \times 14 \times 12$	Bottleneck	4	128	1	2
$128 \times 7 \times 6$	Bottleneck	2	128	2	1
$128 \times 7 \times 6$	Conv 1×1	–	512	1	1
$512 \times 7 \times 6$	GDConv 7×6	–	512	1	1
$512 \times 1 \times 1$	Linear (PW 1×1)	–	128	1	1

When each Bottleneck is expanded into its three constituent sub-layers (PW1, DW, PW2), the full model comprises **50 layers** (L0–L49). Figure 1 visualizes the complete architecture, and Figure 2 illustrates the internal structure of the first Bottleneck block.



Figure 2: Internal structure of MobileFaceNet Bottleneck Block 0 (PW1 $\rightarrow$ DW $\rightarrow$ PW2).

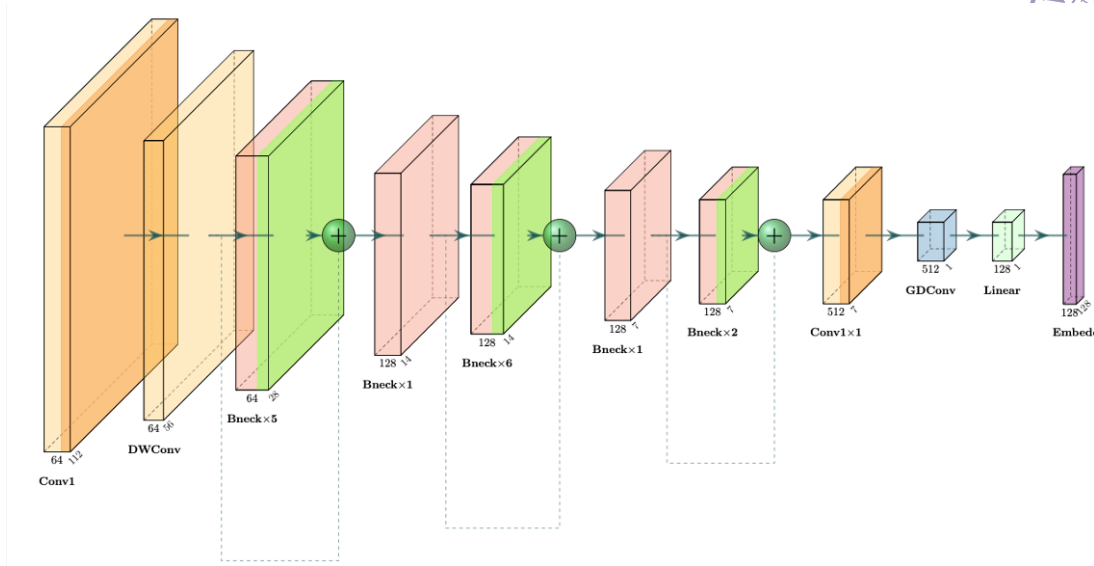
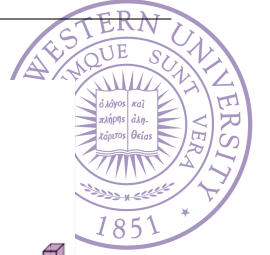


Figure 1: MobileFaceNet architecture visualization.

2.2 Precision Selection

We evaluated four precision formats — PyTorch float32, C float32, C INT16, and C INT8 — by comparing pairwise cosine similarity scores against the PyTorch reference on a face verification task.

Table 2: Precision Comparison Summary

Format	Max Error vs. PyTorch	Verdict
C Float32	≈ 0	Reference
C INT16	< 0.003 (most pairs: OK)	Selected
C INT8	Up to 0.34 (many pairs: WARN)	Rejected

C Float32 closely matches PyTorch, confirming the floating-point implementation is correct. C INT16 (Q10 fixed-point) introduces only negligible error across most pairs, while C INT8 exhibits significantly larger deviations, particularly on negative pairs where the similarity scores are already close to zero. We therefore select **INT16 Q10** as the final representation, as it provides the best balance between hardware efficiency and numerical accuracy.

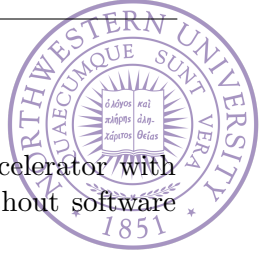
2.3 Model Evaluation

To validate the selected INT16 Q10 model, we measured intra-person and inter-person cosine similarity on two datasets. Results are summarized in Table 3.

Table 3: Face Verification Evaluation (C INT16 Q10)

Dataset	Intra avg	Inter avg	Gap
Original (6 persons, 3 images each)	0.5575	0.0673	+0.4903
Self-prepared (Dan, Elon, Trump, Wil)	0.5270	0.0783	+0.4487

Both datasets show a large separability gap (intra $\gg$ inter), confirming that the INT16 Q10 quantization preserves the model’s discriminative capability with negligible accuracy loss relative to the floating-point reference.



3 Hardware Architecture

Design Positioning: A fixed-point, MobileFaceNet-specific convolution frontend accelerator with ROM-based layer configuration. The design targets 50-layer sequential execution without software intervention between layers.

The architecture is organized into three subsystems:

3.1 Supported Operation Types

- Standard 3×3 Conv (with PReLU)
- 3×3 Depthwise Conv (with PReLU)
- 1×1 Pointwise Conv (with/without PReLU)
- 7×6 Global Depthwise Conv (linear7, no PReLU)
- Residual Shortcut Add

3.2 Datapath

The datapath performs the core convolution computation:

1. **Sliding Window Buffer** (`mfn_sliding_window`): 9-entry pixel buffer providing 3×3 patch or 9-channel slice to the MAC array.
2. **MAC Array** (`mfn_mac_array`): 9 parallel 16-bit signed multipliers, producing a 40-bit dot product per cycle.
3. **Activation Module** (`mfn_activation`): 2-stage pipeline. Stage 1: Q20 $\rightarrow$ Q10 right-shift + int16 clamp. Stage 2: PReLU multiplication + residual add + final clamp.

Fixed-point format summary:

Table 4: Fixed-Point Format

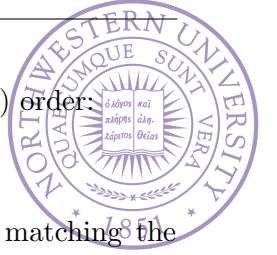
Data	Format	Width
Activation	Q10	int16
Weight	Q10	int16
Bias	Q20	int32
Accumulator	Q20	int40
PReLU alpha	Q10	int16

3.3 Control Path

- **Controller FSM** (`mfn_controller`): Sequences layer execution via a nested loop: `layer` $\rightarrow$ `y,x` $\rightarrow$ `c.out` $\rightarrow$ `c.in`.
- **Address Generator** (`mfn_addr_gen`): 2-stage pipelined HWC address computation.
Stage 1: `pixel_offset` = `y` $\times$ `W`+`x`.
Stage 2: `addr` = `pixel_offset` $\times$ `C` + `c` + `buf_base`.
- **Layer Config ROM**: 50×64 -bit entries, one per layer, loaded via `$readmemh`. Encodes all layer parameters (see Section 4).

3.4 Memory Subsystem

- **Feature Map SRAM**: 1 MB, 20-bit address, 16-bit word width. Partitioned into three ping-pong buffers (B0/B1/B2), each 344,064 words (= $64 \times 56 \times 48 \times 2$, the maximum feature map size).
- **Weight ROM**: 20-bit address, $9 \times$ int16 per read cycle.
- **Bias ROM**: 14-bit address, int32 Q20 output.
- **PReLU ROM**: 14-bit address (shared with Bias ROM index), int16 Q10 alpha per output channel.



HWC Memory Layout. Feature maps are stored in HWC (Height-Width-Channel) order:

$$\text{addr} = (y \times W + x) \times C + c + \text{buf_base}$$

This ensures all channels at a given spatial position (y, x) are stored contiguously, matching the sliding-window access pattern and enabling sequential burst reads with no address striding.

Triple-Buffer Residual Management. Each layer config specifies `rd_buf` and `wr_buf`. The residual buffer is computed as:

$$\text{res_buf} = 3 - \text{rd_buf} - \text{wr_buf}$$

Since buffer indices $\{0, 1, 2\}$ always sum to 3, this guarantees no conflicts without any stall logic.

4 Layer Configuration ROM

Each layer is described by a single 64-bit configuration word:

Table 5: 64-bit Config Word Fields

Field	Bits	Description
<code>in_ch</code>	[9:0]	Input channel count (up to 512)
<code>out_ch</code>	[19:10]	Output channel count
<code>w</code>	[26:20]	Feature map width
<code>h</code>	[33:27]	Feature map height
<code>stride</code>	[35:34]	Stride (1 or 2)
<code>has_prelu</code>	[36]	Apply PReLU activation
<code>is_res</code>	[37]	Add residual shortcut
<code>is_pw</code>	[38]	Pointwise (1×1) conv
<code>is_dw</code>	[39]	Depthwise conv
<code>wgt_base</code>	[59:40]	Weight ROM base address (20-bit)
<code>rd_buf</code>	[61:60]	SRAM read buffer select (0/1/2)
<code>wr_buf</code>	[63:62]	SRAM write buffer select (0/1/2)

The `is_dw=1`, `is_pw=1` combination (normally impossible) is repurposed as the Global DW mode signal for the linear7 layer, requiring no additional config bits.

Table 6: Selected Layer Config Examples

Layer	Hex	in	out	w	h	s	dw	pw	res	rd→wr
L0 Conv1	4000001b86010003	3	64	96	112	2	0	0	0	0→1
L1 DW1	9006c095c3010040	64	64	48	56	1	1	0	0	1→2
L2 PW1	60090055c3020040	64	128	48	56	1	0	1	0	2→1

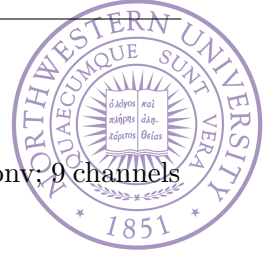
Example Config Words: All 50 config words are generated by `gen_hex.py` and loaded via `$readmemh` at simulation time.

5 Controller FSM & Execution Order

5.1 FSM States

The controller executes the following state sequence per layer:

1. **LOAD_CFG:** Read 64-bit config from ROM at `layer_idx`.



2. **WAIT_CFG**: Decode all fields; compute `wgt_step`.
3. **INIT_PSUM**: Load bias from Bias ROM into accumulator.
4. **FETCH_PIXELS**: Load 9 values into sliding window (spatial neighbors for DW/Conv; 9 channels for PW).
5. **CALC_PSUM**: 9-MAC parallel dot product, accumulate PSUM.
6. **MAC_FLUSH**: Drain 2-cycle pipeline.
7. **CH_IN_LOOP**: If `c_in` not done $\rightarrow$ back to **FETCH_PIXELS**.
8. **READ_RESIDUAL**: If `is_res=1`, read shortcut from `res_buf`.
9. **WRITE_BACK**: Q20 $\rightarrow$ Q10, clamp, PReLU, (+residual), write to SRAM.
10. **SPATIAL_LOOP**: If (x, y) not done $\rightarrow$ back to **INIT_PSUM**.
11. **NEXT_LAYER**: `layer_idx++`, check done $\rightarrow$ IDLE or **LOAD_CFG**.

5.2 Nested Loop Structure

The hardware executes the following nested loop autonomously:

5.3 Pointwise Conv Optimization

For 1×1 PW conv (`is_pw=1`), the sliding window slides over *channel dimension* rather than spatial. Nine input channels are read at once and fed into all 9 MAC lanes simultaneously, greatly improving throughput for PW-heavy layers.

6 Bottleneck Execution

A Bottleneck block is executed as **three consecutive layer iterations** in hardware, each reading a different Config ROM entry:

1. **PW1** (`is_pw=1`, `has_prelu=1`): Channel expansion.
2. **DW** (`is_dw=1`, `has_prelu=1`): Spatial filtering, stride 1 or 2.
3. **PW2** (`is_pw=1`, `has_prelu=0`): Linear channel projection. If `is_res=1`: add shortcut from `res_buf` before write-back.

The residual condition is: `stride = 1 AND in_ch = out_ch`.

Table 7: Sliding Window Behavior: DW vs PW

	3×3 Depthwise	1×1 Pointwise
Window content	9 spatial neighbors	9 input channels
Slides over	spatial (y, x)	channel <code>c_in</code>
<code>c_in</code> loop	$= c_{out} (1:1)$	$\text{ceil}(C/9)$ steps
Address jump	3×3 neighborhood	sequential +1

7 Global Depthwise Conv (linear7)

The final global depthwise layer uses a 7×6 kernel (42 taps), but the MAC array only has 9 lanes. The solution splits the 42 taps into 5 groups of 9:

- Group 0: tap 0–8 $\rightarrow$ `psum0`
- Group 1: tap 9–17 $\rightarrow$ `psum1`
- Group 2: tap 18–26 $\rightarrow$ `psum2`
- Group 3: tap 27–35 $\rightarrow$ `psum3`
- Group 4: tap 36–41 + 3 zeros $\rightarrow$ `psum4`



$$\text{output} = \sum_{i=0}^4 \text{psum}_i \quad \text{per channel,} \times 512 \text{ channels}$$

Key Insight: The MAC array, sliding window, and accumulator are *unchanged*. Only the controller runs 5 MAC passes instead of 1. Zero hardware redesign needed. The `is_dw=1`, `is_pw=1` config encoding (normally invalid) is repurposed as the Global DW mode signal.

8 Systolic Array Implementation

8.1 Current Progress

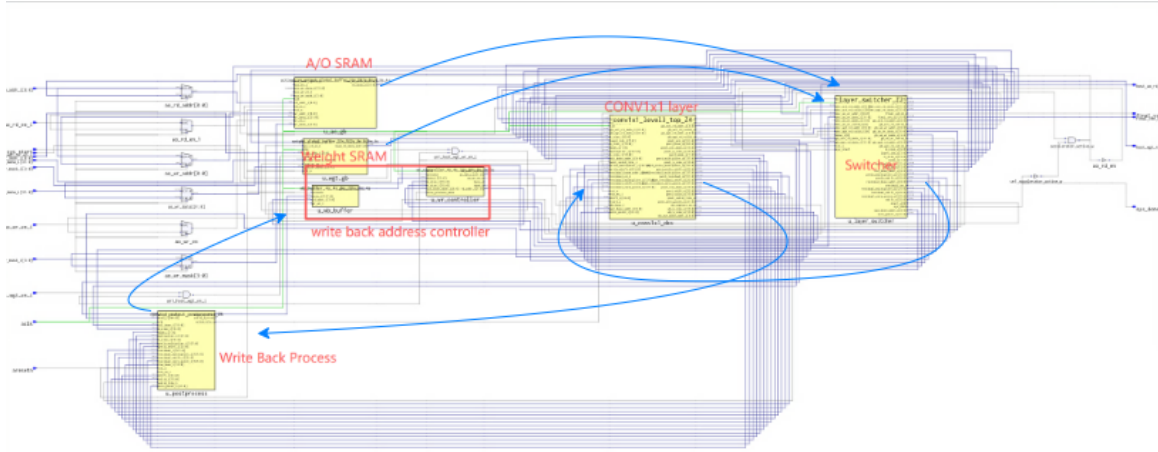


Figure 3: System-level architecture of the current systolic-array accelerator

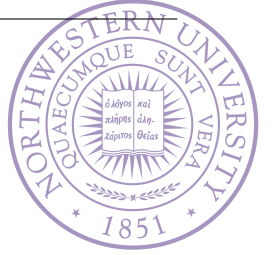
Figure 3 shows the current closed-loop accelerator system. The design is organized around shared global buffers, a layer switcher, one layer device, a shared postprocess unit, and a shared write-back path.

1. **A/O SRAM:** Stores input activations and intermediate/final layer outputs.
2. **Weight SRAM:** Stores quantized weights for all supported layers.
3. **Switcher:** Selects the active layer device and routes SRAM read/write channels.
4. **Layer device:** Currently supports the `conv1x1` datapath, including preload, systolic-array computation, residual read, and tile-level control.
5. **Postprocess:** Converts 32-bit partial sums to 8-bit activations. It supports bias, requantization, residual add, and PReLU modes.
6. **Write-back buffer / WR controller:** Packs processed 8-bit outputs into SRAM words and generates NHWC write-back addresses.

The current implementation has completed closed-loop verification for the `conv1x1` layer. The global buffers are not instantiated inside the layer device; instead, they are shared at the system level. The `conv1x1` device is also decoupled from postprocess and write-back. It only outputs a raw MAC tile and tile metadata through a common interface. Therefore, future layer devices can reuse the same switcher, postprocess unit, and write-back path. Figure 4 shows the preliminary synthesis timing result.

8.2 Next Step

The next step is to add more layer devices, starting from depthwise convolution and regular convolution. Each new layer will implement its own internal controller and computation datapath, while



Timing Summary			
Clock Name (clock_name)	Req Freq (req_freq)	Est Freq (est_freq)	Slack (slack)
top1ack	103.7 MHz	88.1 MHz	-1.702
Detailed report	Timing Report View		

Figure 4: Preliminary synthesis timing result of the current system

reusing the existing switcher, shared postprocess unit, and write-back path. After that, the full MobileFaceNet layer schedule can be mapped into the layer configuration ROM and verified against the C golden model.

9 Verification

9.1 Verification Flow

PyTorch (float32) $\rightarrow$ C-model (INT16 Q10) $\rightarrow$ RTL simulation $\rightarrow$ Bit-level comparison

The C-model serves as the fixed-point golden reference. RTL output is compared against C-model output layer by layer.

9.2 Layer-Level Verification Results

All 50 layers achieved 100.00% exact match (ExactMatch%, $\pm 1\%$, MeanErr, MaxErr):

Table 8: Verification Summary (Selected Layers)

Layer	Exact%	$\pm 1\%$	MeanErr	MaxErr
L00	100.00%	100.00%	0.000	0
L10	100.00%	100.00%	0.000	0
L20	100.00%	100.00%	0.000	0
L30	100.00%	100.00%	0.000	0
L40	100.00%	100.00%	0.000	0
L49	100.00%	100.00%	0.000	0
ALL	100.00%	100.00%	0.000	0

Status: **ALL PASS**

9.3 End-to-End Embedding Verification

RTL-generated 128-d embeddings were compared against C-model embeddings on the self-prepared dataset:

Table 9: RTL vs C-Model Embedding Exact Match

Image	Match	MaxDiff	MeanAbsErr
dan_1.jpg	128/128	0	0.0000
elon_1.jpg	128/128	0	0.0000
trump_1.jpg	128/128	0	0.0000
wil_1.jpg	128/128	0	0.0000
(all 13 images)	128/128	0	0.0000



Table 10: Similarity Summary Comparison: C Fixed-Point vs RTL

Metric	C Fixed	RTL
Intra avg	0.5270	0.5270
Inter avg	0.0783	0.0783
Gap	+0.4487	+0.4487

RTL output is bit-exact with the C-model, confirming the hardware correctly implements the fixed-point datapath.

10 TODO & Future Work

Table 11: Project Roadmap

Priority	Phase	Task
Now	Synthesis	Run Design Compiler; check timing report at 100 MHz
Now	Synthesis	Fix critical path: addr_gen 2-stage pipeline + activation PReLU register
Near	System Integration	Replace Weight ROM → SRAM + AXI-Lite write interface
Near	System Integration	AXI-Stream DMA interface for feature map I/O
Near	Verification	FPGA integration on Zynq
Mid	Datapath Upgrade	Replace 1×1 PW datapath with Systolic Array
Long	Backend	Innovus ASIC backend flow (floorplan → routing → GDSII)

10.1 Synthesis Critical Paths

Three predicted critical paths in the current RTL:

1. **addr_gen**: Two sequential multiplications ($y \times W + x$, then $\times C + c$) in one cycle → split into 2-stage pipeline.
2. **mfn_activation**: PReLU multiplier between two combinational stages → insert pipeline register after multiplier.
3. **mfn_controller**: Large FSM combinational block → register outputs.

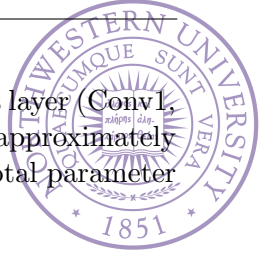
11 Deliverables

- Top-level architecture and block diagrams of the MobileFaceNet frontend accelerator
- RTL design of the main accelerator modules, including convolution datapath, control FSM, address generator, and memory subsystem
- Fixed-point C reference model and bit-level functional verification against PyTorch golden output
- Simulation results with 100% exact match across all 50 layers of MobileFaceNet (112×96 input)
- Synthesis results and timing analysis targeting 100 MHz (Design Compiler)
- Runtime weight update support via writable weight SRAM and AXI-Lite interface
- Final report and presentation

12 Response to Midterm Feedback

12.1 Grayscale Input Reduction

The suggestion to reduce the input from RGB to grayscale is noted. While converting the input from 3-channel RGB to single-channel grayscale would reduce the weight count of the first convolutional



layer by a factor of 3, the practical impact on the overall model size is negligible. The first layer (Conv1, $3 \times 64 \times 3 \times 3$) contains only 1,728 weights, whereas the total model weight count is approximately 1,000,000. The reduction from 1,728 to 576 weights represents less than **0.1%** of the total parameter budget, as shown in Table 12.

Table 12: Weight Distribution Across Major Layer Groups

Layer Group	Weight Count	Fraction
Conv1 (RGB, $3 \times 64 \times 3 \times 3$)	1,728	0.17%
Conv1 (GS, $1 \times 64 \times 3 \times 3$)	576	0.06%
All Bottleneck layers (L2–L46)	~890,000	~89%
Conv2 ($128 \times 512 \times 1 \times 1$)	65,536	6.5%
linear1 ($512 \times 128 \times 1 \times 1$)	65,536	6.5%
linear7 (GDConv $512 \times 7 \times 6$)	21,504	2.1%

Furthermore, the existing model weights were trained on RGB face images. Switching to grayscale input would require **retraining the entire model** from scratch on a grayscale face dataset, as the learned weight distributions in Conv1 are specific to three-channel input statistics. Grayscale conversion also discards color information (skin tone, makeup, lighting cues) that may contribute to face verification accuracy. Given the minimal memory savings and the significant retraining cost, we conclude that grayscale reduction is not beneficial for this design.

12.2 Task Definition: Face Verification, Not Classification

MobileFaceNet performs **face verification** determining whether two face images belong to the same person, rather than face classification (assigning an identity label from a fixed set) or face detection (localizing a face region within an image).

The model takes a pre-cropped, aligned face image as input and produces a **128-dimensional embedding vector**. Two images are considered to belong to the same person if the cosine similarity between their embeddings exceeds a threshold:

$$\text{same person} \iff \frac{\mathbf{e}_1 \cdot \mathbf{e}_2}{\|\mathbf{e}_1\| \|\mathbf{e}_2\|} > \tau$$

The model does not maintain an identity database or classify among a closed set of known persons. Instead, it maps any face image into a metric space where intra-person distances are small and inter-person distances are large.

In our evaluation, the INT16 Q10 model achieves an intra-person similarity average of 0.5270 and an inter-person average of 0.0783, yielding a separability gap of **+0.4487**, which confirms the embedding space is well-structured for reliable verification.

12.3 Hardware Verification

We have implemented and verified the complete RTL at two levels: **layer-level** verification against the fixed-point C-model golden reference, and **end-to-end** embedding verification on real face images shown in Section 9.

Verified Modules. The following RTL modules have been individually exercised through the top-level simulation:

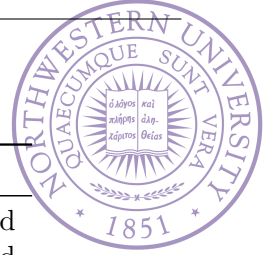


Table 13: Hardware Modules and Verification Status

Module	Function	Status
<code>mfn_controller</code>	Main FSM, nested loop scheduling	✓ Verified
<code>mfn_addr_gen</code>	HWC address generation, ping-pong	✓ Verified
<code>mfn_sliding_window</code>	3×3 pixel buffer, zero-pad	✓ Verified
<code>mfn_mac_array</code>	9-lane 16-bit parallel MAC	✓ Verified
<code>mfn_activation</code>	Q20→Q10, clamp, PReLU, residual	✓ Verified
<code>mfn_layer_config_rom</code>	64-bit config word decode	✓ Verified
<code>mfn_frontend_top</code>	Full system integration (L0–L49)	✓ Verified

Layer-Level Results. All 50 layers (L0–L49) were verified against C-model golden vectors. Every layer achieved **100.00% exact match** with zero mean error and zero maximum error across all output pixels.

End-to-End Results. The RTL-generated 128-dimensional embeddings were compared against C-model embeddings for 13 face images (Dan, Elon, Trump, Wil). All images achieved **128/128 exact match** (MaxDiff = 0, MeanAbsErr = 0.0000), and the cosine similarity matrix was identical between RTL and C-model.

Pending Verification. Synthesis-level (gate-level) simulation and FPGA integration remain as future work, as described in the project roadmap (Section 10).

References

- [1] Sheng Chen, Yang Liu, Xiang Gao, and Zhen Han. Mobilefacenets: Efficient cnns for accurate real-time face verification on mobile devices, 2018.
- [2] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1):127–138, 2017.
- [3] Linjun SUN, Weijun LI, Xin NING, Liping ZHANG, Xiaoli DONG, and Wei HE. Gradient-enhanced softmax for face recognition. *IEICE Transactions on Information and Systems*, E103.D(5):1185–1189, 2020.